

Roll Number

--	--	--	--	--	--	--

0	0	0	0	0	0	0
1	1	1	1	1	1	1
2	2	2	2	2	2	2
3	3	3	3	3	3	3
4	4	4	4	4	4	4
5	5	5	5	5	5	5
6	6	6	6	6	6	6
7	7	7	7	7	7	7
8	8	8	8	8	8	8
9	9	9	9	9	9	9

The OMR Sheet will be computer checked. Fill the circles completely and dark enough for proper detection. Use Dark HB pencil or ballpen (black or blue) for marking. While erasing, erase properly.

Avoid Improper Marking



Partially Filled



Lightly filled



Analysis & Design of Algorithms 2026 — Quiz 3

Duration: 1 hour

Total Marks: 20

Name: _____

Roll No.: _____

Problem 1. (10 points) The Delhi Jal Board (DJB) is planning emergency water supply for the upcoming summer season in Old Delhi. The water distribution network in the Chandni Chowk area consists of a set of junctions connected by pipes of various capacities - the capacity of a pipe is the volume of water per hour (in kilolitres) that the pipe can carry, and water can flow in either direction through any pipe.

The DJB has a single water pumping station at Tis Hazari (node s) and needs to supply water to the Fatehpuri Masjid community tanker station (node t). They want to route water along a **single path** from s to t (since using multiple paths simultaneously requires expensive pressure-balancing equipment they cannot currently afford).

The ‘flow’ along any single path is limited by the narrowest pipe on that path—i.e., the pipe with the smallest capacity determines the maximum hourly flow for the entire path.

Given the network as an undirected weighted graph $G = (V, E, c)$ where $c(e) \in \{1, 2, \dots, U\}$ is the capacity of pipe e : Design an algorithm that runs in $O((|V| + |E|) \log U)$ time to find the **maximum** hourly supply of water possible subject to the conditions above.

(a) Give a 3-4 sentence description of the high level idea of your algorithm. **[2 marks]**

Solution. We use **binary search on the bottleneck capacity** m . For a candidate value m , we construct the subgraph G_m consisting of all edges with capacity $\geq m$ and check whether s can reach t in G_m using BFS or DFS, which runs in $O(|V| + |E|)$ time.

- If s reaches t in G_m , then a path with bottleneck $\geq m$ exists; we search higher.
- Otherwise, we search lower.

Since the search space is $\{1, 2, \dots, U\}$, binary search performs $O(\log U)$ iterations, giving $O((|V| + |E|) \log U)$ total.

Rubric : +1 for just mentioning binary search, +1 for properly saying how to use BFS/DFS (no score for this part unless you mention the $\geq m$ thing explicitly)

(b) Write the pseudocode on the basis of the above intuition. If the two don’t match, you will get zero for both. **[6 marks]**

Algorithm 1 MAXBOTTLENECKPATH($G = (V, E, c)$, s , t)

```
1:  $lo \leftarrow 1$ ;  $hi \leftarrow U$ ;  $ans \leftarrow 0$ 
2: while  $lo \leq hi$  do
3:    $mid \leftarrow \lfloor (lo + hi)/2 \rfloor$ 
4:   if REACHABLE( $G$ ,  $s$ ,  $t$ ,  $mid$ ) then
5:      $ans \leftarrow mid$ 
6:      $lo \leftarrow mid + 1$ 
7:   else
8:      $hi \leftarrow mid - 1$ 
9:   end if
10: end while
11: return  $ans$ 
```

Algorithm 2 REACHABLE($G = (V, E, c)$, s , t , $threshold$)

```
1: Initialise  $visited[v] \leftarrow \mathbf{false}$  for all  $v \in V$ 
2:  $Q \leftarrow$  empty queue; enqueue  $s$ ;  $visited[s] \leftarrow \mathbf{true}$ 
3: while  $Q$  is not empty do
4:    $u \leftarrow Q.dequeue()$ 
5:   if  $u = t$  then
6:     return true
7:   end if
8:   for each edge  $(u, w) \in E$  do
9:     if  $c(u, w) \geq threshold$  and  $!visited[w]$  then
10:       $visited[w] \leftarrow \mathbf{true}$ 
11:       $Q.enqueue(w)$ 
12:    end if
13:  end for
14: end while
15: return false
```

(c) Argue runtime of the above pseudocode.

[2 marks]

- The binary search over $\{1, \dots, U\}$ executes $O(\log U)$ iterations.
- Each call to REACHABLE performs a BFS that visits every vertex at most once and examines every edge at most once (via adjacency lists), running in $O(|V| + |E|)$.
- Total time: $\mathcal{O}(\log U) \times \mathcal{O}(|V| + |E|) = \mathcal{O}((|V| + |E|) \log U)$.

Rubric

- (a) 1 mark for Identifying binary search on the capacity value.
1 mark for Identifying the reachability/connectivity check on the filtered subgraph (edges with capacity $\geq m$) as the subroutine.
- (b) 1 mark for Correct binary search structure (**lo**, **hi**, loop, midpoint computation).
1 mark for Correct update logic (increase **lo** on success, decrease **hi** on failure).
1 mark for Correctly filtering edges by $c(e) \geq threshold$.
1 mark for Correct BFS/DFS implementation for reachability.
1 mark for Correctly tracking and returning the best feasible value.
1 mark for Overall consistency: pseudocode matches high-level idea; handles edge cases (e.g., no path $\Rightarrow$ returns 0).

Note: If (a) and (b) contradict each other, both receive 0 (per exam instructions).

- (c) 1 mark for Correctly stating $\mathcal{O}(\log U)$ iterations for binary search.
1 mark for Correctly stating $\mathcal{O}(|V| + |E|)$ per iteration and combining to get $\mathcal{O}((|V| + |E|) \log U)$.

Problem 1: Alternate Solution (Dijkstra-Style)

$\mathcal{O}(|E| \log |V|)$

Key Insight. Instead of binary searching on capacity, we can adapt Dijkstra’s algorithm. In Dijkstra’s, we relax edges to find the shortest (minimum-sum) path. Here, we want the *maximum bottleneck* path, so we replace the “min-sum” with “max-min”: we greedily expand the vertex whose best bottleneck value is largest, using a max-heap.

(a) High-Level Idea

[2 marks]

We maintain, for every vertex v , a value $bn[v]$ representing the best (maximum) bottleneck of any path found so far from s to v . Initially $bn[s] = \infty$ (the source has no constraining edge) and $bn[v] = -\infty$ for all other vertices. We use a **max-priority queue**: at each step, extract the vertex u with the largest $bn[u]$ and relax its neighbours — for each neighbour

w via edge (u, w) , the bottleneck through u is $\min(bn[u], c(u, w))$; if this exceeds $bn[w]$, we update $bn[w]$. This is correct because, just as in Dijkstra's, once a vertex is extracted its value is optimal (the greedy "best-first" expansion ensures no future path can improve it). We stop when t is extracted.

(b) Pseudocode

[6 marks]

Algorithm 3 MAXBOTTLENECKDIJKSTRA($G = (V, E, c)$, s , t)

```

1:  $bn[v] \leftarrow -\infty$  for all  $v \in V$ 
2:  $bn[s] \leftarrow +\infty$ 
3:  $finalised[v] \leftarrow \mathbf{false}$  for all  $v \in V$ 
4:  $H \leftarrow$  max-priority queue; insert  $(s, +\infty)$ 
5: while  $H$  is not empty do
6:    $(u, b_u) \leftarrow H.EXTRACTMAX()$ 
7:   if  $finalised[u]$  then
8:     continue ▷ skip stale entries
9:   end if
10:   $finalised[u] \leftarrow \mathbf{true}$ 
11:  if  $u = t$  then
12:    return  $b_u$ 
13:  end if
14:  for each edge  $(u, w) \in E$  do
15:     $newbn \leftarrow \min(b_u, c(u, w))$ 
16:    if  $newbn > bn[w]$  then
17:       $bn[w] \leftarrow newbn$ 
18:       $H.INSERT(w, newbn)$ 
19:    end if
20:  end for
21: end while
22: return 0 ▷  $t$  unreachable

```

(c) Runtime Analysis

[2 marks]

- Each vertex is finalised at most once. Each edge (u, w) is relaxed at most once (when u is finalised), leading to at most $|E|$ INSERT operations and at most $|E|$ EXTRACTMAX operations on the heap.
- Each heap operation takes $\mathcal{O}(\log |E|) = \mathcal{O}(\log |V|)$ time (since $|E| \leq |V|^2$).
- Total time: $\mathcal{O}(|E| \log |V|)$.

Remark. This solution runs in $\mathcal{O}(|E| \log |V|)$ rather than $\mathcal{O}((|V| + |E|) \log U)$. The two are incomparable in general: the binary-search approach is better when U is small, while the Dijkstra approach is better when U is large relative to $|V|$. So ideally, this solution should fetch a **zero** since the runtime of this solution may be much much bigger (if $U \ll n$). However, I am allowing full credit this time. Please be careful about such things in quiz 5 and endsem.

Rubric for Alternate Solution.

(a) High-Level Idea [2 marks]

- (1 mark) Identifying the Dijkstra-style greedy approach: maintain best bottleneck value per vertex, expand greedily using a max-priority queue.
- (1 mark) Correct relaxation rule: $bn[w] \leftarrow \max(bn[w], \min(bn[u], c(u, w)))$, i.e. "max of min" instead of Dijkstra's "min of sum".

(b) Pseudocode [6 marks]

- (1 mark) Correct initialisation: $bn[s] = +\infty$, all others $-\infty$.

- (1 mark) Using a **max**-priority queue (not min).
- (1 mark) Correct extract-max loop with finalisation / visited check to avoid reprocessing.
- (1 mark) Correct relaxation step: computing $\min(bn[u], c(u, w))$ and updating $bn[w]$ only if it improves.
- (1 mark) Inserting updated vertices into the heap (or using decrease-key).
- (1 mark) Overall correctness: pseudocode matches intuition from (a); correctly returns $bn[t]$ or handles unreachability.

Note: If (a) and (b) contradict each other, both receive 0 (per exam instructions).

(c) **Runtime Analysis** [2 marks]

- (1 mark) Correctly bounding the number of heap operations at $\mathcal{O}(|E|)$ (each edge relaxed at most once).
- (1 mark) Correctly stating each heap operation is $\mathcal{O}(\log |V|)$ and combining to get $\mathcal{O}(|E| \log |V|)$.

Common deductions for the Dijkstra-style solution:

- Using a min-heap instead of max-heap: -1 from (b).
- Missing or incorrect initialisation ($bn[s] \neq +\infty$, or others $\neq -\infty$): -1 from (b).
- Using sum instead of min for the relaxation: -1 from (a) and -1 from (b).
- Claiming $\mathcal{O}(|V| \log |V|)$ without justification: -1 from (c).

Problem 2. The Delhi Metro Rail Corporation (DMRC) operates a network of stations connected by direct rail links. Due to recent security concerns, the DMRC needs to identify *critical stations*—stations whose closure would disconnect the network, leaving some stations unreachable from others.

The network is modelled as a connected undirected graph $G = (V, E)$, where each vertex represents a station and each edge represents a direct rail link. We say that a vertex v is a *critical station* if the graph obtained by removing v and all edges incident to v —that is, the induced subgraph $G[V \setminus \{v\}]$ —is disconnected. You may assume that the network G is connected.

Suppose we run DFS on G from some source vertex r and obtain a DFS tree T rooted at r . For each vertex v , let $\text{arr}[v]$ denote its arrival time and $\text{dbe}[v]$ denote the discovery time of the deepest ancestor reachable from the subtree of v via a back edge.

(a) Can a leaf of T be a critical station of G ? Justify your answer. **[3 marks]**

Solution. No. Let v be a leaf of the DFS tree T .

Proof. Since v is a leaf in T , the subtree T_v consists of v alone; v has no children in T . Every edge incident to v is either:

- (1) the tree edge connecting v to its parent, or
- (2) a back edge connecting v to an ancestor.

No other vertex in G depends on v for its connectivity, because v does not lie on the unique tree path between any two other vertices.

The above is sufficient for full score. But here is the formal proof. $G[V \setminus \{v\}]$ remains connected: for any two vertices $x, y \neq v$, the tree path from x to y in T either avoids v entirely (in which case it still exists), or would pass through v only if v were an internal vertex (but v is a leaf, so no tree path between two other vertices passes through it).

Therefore, removing v cannot disconnect G , and v is *not* a critical station.

(b) State a *necessary* and *sufficient* condition, in terms of the structure of T , for the root r to be a critical station of G . Justify your answer. **[3.5 marks]**

Solution.

Claim. The root r is a critical station if and only if r has **two or more children** in the DFS tree T .

Proof.

Two or more children implies Critical : Let u and w be two distinct children of r in T . In an undirected DFS, there are no cross edges—every non-tree edge is a back edge connecting a vertex to one of its ancestors. Therefore, there is no edge between any vertex in T_u (the subtree rooted at u) and any vertex in T_w . The only way vertices in T_u connect to vertices in T_w is through r . Removing r therefore disconnects T_u from T_w , so r is critical.

If r has exactly one child or zero child, then r is not critical. :

If r has a single child u , then T_u contains all vertices of G except r . Every edge in $G[V \setminus \{r\}]$ is present, and T_u already spans all of $V \setminus \{r\}$, so the graph remains connected. (If r has zero children then $|V| = 1$ and the question is vacuous.)

- (c) State a *necessary* and *sufficient* condition, in terms of arr and dbe , for a non-root internal vertex v of T to be a critical station of G . Justify your answer. **[3.5 marks]**

Solution.

Claim. A non-root internal vertex v of T is a critical station if and only if v has some child w in T such that

$$dbe[w] \geq arr[v].$$

(Equivalently, no back edge from the subtree T_w reaches a *proper* ancestor of v .)

Proof.

($\Rightarrow$) **If $\exists$ child w with $dbe[w] \geq arr[v]$, then v is critical.**

Since $dbe[w] \geq arr[v]$, the deepest ancestor reachable from T_w via back edges is v itself (or a descendant of v). This means there is *no* edge from any vertex in T_w to any proper ancestor of v . Every path from a vertex in T_w to a vertex outside T_v must therefore pass through v . Removing v disconnects w (and its subtree) from the parent of v , so $G[V \setminus \{v\}]$ is disconnected.

($\Leftarrow$) **If $dbe[w] < arr[v]$ for every child w , then v is not critical.**

For every child w of v , the condition $dbe[w] < arr[v]$ means there exists a back edge from some vertex in T_w to a proper ancestor of v . Therefore, even after removing v , every vertex in each child's subtree can reach an ancestor of v via such a back edge, and the graph $G[V \setminus \{v\}]$ remains connected.

Remarks. You will be awarded full score even if you wrote the above using phrases like ‘deepest back edges’ and ‘proper ancestors’ etc. instead of dbe , arr .

Rurbic.

(a) **Leaf as critical station** [3 marks]

- (1 mark) Correct answer: **No**.
- (2 marks) Justification: a leaf has no subtree depending on it; removing it cannot separate any other vertices. Partial credit (1 mark) for incomplete or vague reasoning.

(b) **Condition for the root** [3.5 marks]

- (1.5 marks) Correct condition: root has ≥ 2 children in T .
- (1 mark) Forward direction: ≥ 2 children implies disconnection (no cross edges in undirected DFS between sibling subtrees).
- (1 mark) Reverse direction: single child means root is not critical (remaining graph spans all non-root vertices).

(c) **Condition for non-root internal vertex** [3.5 marks]

- (1.5 marks) Correct condition: $\exists$ child w with $dbe[w] \geq arr[v]$.
- (1 mark) Forward direction: condition holds $\Rightarrow$ subtree T_w is disconnected from ancestors upon removing v .
- (1 mark) Reverse direction: condition fails for all children $\Rightarrow$ back edges maintain connectivity.